INFRAPULSE

Implementation Backlog, Acceptance Criteria and Test Plan

SDG 9 — Industry, Innovation and Infrastructure

DEVELOPMENT TASKS & QA DOCUMENT

Build-ready specification for an AI-assisted development workflow

Core proposition: Infrastructure availability does not guarantee reliable service. InfraPulse combines availability, service-performance evidence, accessibility, and population impact to help identify reliability gaps and compare interventions.

Version 1.0 | Prepared 9 October 2026 | Hackathon MVP with production-minded extension path

1. Delivery strategy

Build the smallest end-to-end product first: demo data → scoring API → village map/list → village detail → gap detection → priority ranking → scenario simulator → export → deployment. Do not begin optional AI or advanced forecasting before this path works.

2. Definition of done

- Feature is implemented, reviewed and usable in the deployed app.
- Validation and error states are implemented; no silent failures.
- Tests cover happy path, boundary cases and missing/invalid data.
- Relevant documentation and screenshots are updated.
- Simulated data is visibly labelled in UI and exports.
- No secret keys or personal information are committed.
- All acceptance criteria for the task are checked.

3. Prioritized backlog

ID / Priority	Task	Acceptance criteria / deliverable
T01 P0	Create repository and developer setup	Frontend/backend folders; README prerequisites; .env.example; ignore secrets/build artifacts; fresh clone setup tested.
T02 P0	Define canonical schema and data dictionary	Village and indicator schemas documented; required fields, units, evidence types and provenance specified.
T03 P0	Create small demo dataset	At least 20 fictional/demo village records with coordinates, component indicators and clear simulation labels; no invented values presented as official.
T04 P0	Implement CSV/GeoJSON validation	Reject bad coordinates, duplicate IDs, wrong types and missing required fields; report row-level errors.
T05 P0	Implement normalization functions	Higher/lower-is-better rules; clamps; unit handling; missing/stale/invalid states; unit tests.
T06 P0	Implement component and overall score	Versioned weights; weighted mean; minimum coverage rule; category; formula tests.
T07 P0	Implement availability-reliability gap	Calculate only comparable values; suppress for missing or date mismatch; tests for positive/zero/negative gap.
T08 P0	Implement priority ranking	Configurable factors; deterministic sort; factor-level explanation; tie-breaker tests.
T09 P0	Implement scenario simulator	Validate overrides; recalculate score; compare baseline/scenario; no baseline mutation; tests.
T10 P0	Build API endpoints	Health, summary, list, detail, map, rankings, scenarios, methodology, export; typed schemas and useful errors.
T11 P0	Build app shell and navigation	Responsive layout; pages for overview, map explorer, village detail, ranking, scenario lab, methodology.
T12 P0	Build overview dashboard	KPI cards, score distribution, critical-village list, data freshness/coverage notices; matches API data.
T13 P0	Build map explorer	Village markers/polygons, legend, filter controls, search, synchronized table/list and selected detail.
T14 P0	Build village profile	Overall score, dimension cards, evidence dates, source links, gap, priority factors, missing-data warnings.
T15 P0	Build ranking page	Sort/filter by score and priority; visible factor breakdown; export filtered list.
T16 P0	Build scenario lab	Select village, change dimension assumption, show baseline vs scenario, delta, category and caveat.
T17 P1	Build export	CSV export contains IDs, dimensions, raw/normalized values, score version, data provenance and simulation status.
T18 P1	Build methodology page	Formula, weights, thresholds, missing-data policy, limitations and source links visible in app.
T19 P1	Add field observation workflow	Optional report with dimension, date, text and verification status; aggregate publicly; avoid personal data.
T20 P1	Add deterministic explanation engine	Template explanation based on top score drivers and structured fields; no unsupported claims.
T21 P1	Add optional LLM narration	Only after deterministic explanation works; strict JSON input/output; citations to field IDs; fallback if unavailable.
T22 P1	Deploy frontend/backend	Environment variables, CORS, database migration/seed, health check, production build and live smoke test.

ID / Priority	Task	Acceptance criteria / deliverable
T23 P1	Prepare demo script and screenshots	Two-minute story: detect gap → explain impact → rank → simulate intervention → show evidence/limitations.
T24 P2	Advanced forecasts / live sensors	Only after verified data source, repeatable ingestion and validation plan exist.

4. Suggested work allocation

Role	Primary responsibilities	Must coordinate on
Backend/data	Schema, seed data, validation, scoring, priority, scenarios, API, tests.	API response types and scoring rules.
Frontend/map	App shell, dashboard, map, filters, profile, ranking, simulator.	Shared types, loading/error states and map data.
Research/product	Source register, indicator definitions, demo scenario, methodology, limitations, presentation.	Do not imply simulated metrics are real.
Integration/release	Environment config, deployment, end-to-end tests, README, final demo rehearsal.	API CORS, database and release freeze.

If only one developer is available, complete T01–T16 in order, then T17–T18, then deploy. Do not attempt live data integration until the end-to-end demo is stable.

5. Suggested 24-hour schedule

Elapsed time	Milestone	Exit condition
0–1 h	Scope freeze and repository	MVP checklist, data schema, screen wireframes and ownership fixed.
1–3 h	Data model + demo fixtures	20+ labelled demo villages load and validate.
3–6 h	Scoring domain + unit tests	Scores, coverage, gap and priority tests pass.
6–9 h	API and persistence	List/detail/summary/map/ranking/scenario endpoints return documented schema.
9–13 h	Frontend shell + map	Map, filters, table and village profile work end to end.
13–16 h	Gap, ranking, simulator UI	Core differentiator is demonstrable.
16–18 h	Export and methodology	Download and in-app methodology match calculations.
18–20 h	Integration and deployment	Public URL works; health check and smoke tests pass.
20–22 h	QA and bug fixes	Critical flows pass on a clean browser/session.
22–23 h	Demo/presentation rehearsal	Two-minute walkthrough and fallback screenshots ready.
23–24 h	Release freeze	Only blocker fixes; no new features.

6. Test plan

Unit tests — domain logic

• Normalization: minimum, maximum, in-range, out-of-range, reversed bounds, zero-width bounds and invalid units.

• Missing-data policy: missing indicator, stale indicator, insufficient dimension coverage and no overall score.

• Composite score: known inputs produce exact expected weighted result; weights sum/config validation; rounding only at display layer.

• Gap: availability 90/reliability 50 returns +40; equal values return 0; missing input returns no numeric gap; incompatible periods return warning.

• Priority: factor contributions sum as expected; sort order deterministic; ties use stable ID.

• Scenario: no-op leaves score unchanged; override changes result as expected; invalid values rejected; baseline observations unchanged.

API tests

• Health endpoint returns success and version.

• Unknown village returns 404; malformed request returns 422 with actionable detail.

• Filters return correct subset and consistent totals.

• Map endpoint returns valid GeoJSON and omits invalid geometry safely.

• CSV export columns and rows match active filters.

• Import endpoint reports accepted/rejected counts and does not silently drop invalid rows.

• Scenario endpoint returns baseline, scenario, delta and assumptions.

Frontend end-to-end tests

• Dashboard loads or shows an actionable error if API is unavailable.

• Selecting a village on map and table opens the same profile.

• Filters update map, counts, ranking and export consistently.

• Missing data is shown as “insufficient evidence,” not zero.

• Scenario controls update only scenario output; reset restores baseline.

• Keyboard navigation and table fallback work without relying solely on color.

7. Test fixtures and expected behavior

Fixture	Inputs	Expected result
Complete record	All five dimension scores valid.	Overall score calculated from configured weights; coverage 100%.
Partial record	Three dimensions valid, two missing.	Score calculated only if valid-weight coverage meets threshold; coverage displayed.
Insufficient record	Only one low-weight dimension valid.	No overall score if coverage below threshold; reason shown.
High availability, low reliability	Comparable scores 95 and 52.	Gap = 43 points; flag high availability/low reliability.
Date mismatch	Availability current, reliability stale beyond tolerance.	Numeric gap suppressed or marked low-confidence according to config.
Scenario no-op	No score overrides.	Scenario score equals baseline score.
Scenario improvement	Increase road component score by 20; weight 20%.	If all other inputs/coverage fixed, overall score increases by 4 points before display rounding.
Invalid coordinate	Latitude 120.	Row rejected with coordinate-range error.
Duplicate ID	Two rows share village ID.	Import reports duplicate and follows explicit configured reject/merge policy.

8. Data QA checklist

• Confirm source licensing and attribution requirements before using external data.

• Store source date and observation window for every performance metric.

• Check whether population and infrastructure geometries refer to the same geographic unit.

• Check duplicates, missing coordinates, outliers and impossible values.

• Do not compare percentages, distances, speeds and outage durations without normalization rules.

• Do not combine data from different years without showing a temporal caveat.

• Keep simulated records visually and programmatically distinct from verified records.

• Review at least a small sample against source pages or field observations before presenting real-world claims.

9. Security and release checklist

• API keys only in server-side environment variables.

• CORS restricted to expected origins in deployment.

• File uploads have type and size limits and safe parsing.

• User-provided text is escaped before rendering.

• Public views do not expose reporter identity or precise household location.

• Logs omit secrets and personal report content.

• Database migrations and seed process documented.

• Health endpoint works; deployment logs checked; rollback or known-good build available.

10. Demo script

Time	Action	Narration
0:00–0:20	Open dashboard and map.	“Infrastructure presence is not the same as reliable service.”
0:20–0:45	Select a demo village with a large availability-reliability gap.	“This village is recorded as connected, but the performance evidence is weak. These numbers are clearly marked as demo data.”
0:45–1:05	Open the profile and evidence panel.	“We show the data source, date, confidence and missing evidence instead of hiding uncertainty.”
1:05–1:25	Open priority ranking.	“Priority combines infrastructure deficit, population affected, service criticality and alternatives, with the factor breakdown visible.”
1:25–1:50	Run a road vs broadband scenario.	“This is a scenario comparison based on assumptions, not a promise of causal impact. It shows which intervention deserves further investigation.”
1:50–2:00	Show methodology/source page.	“InfraPulse complements existing GIS and infrastructure programmes with a traceable reliability and decision-support workflow.”

11. AI coding assistant master prompt

Copy the following as the initial instruction to your coding AI. Then implement the task IDs in order and ask it to report changed files and test results after each milestone.

```
You are implementing InfraPulse, a rural infrastructure reliability intelligence platform. Treat
docs/PRD.md, docs/ARCHITECTURE.md and docs/TASKS.md as source of truth. Do not invent live data or present
simulated data as official. Build React + Vite + TypeScript frontend and FastAPI backend. Implement
deterministic, tested scoring, coverage-aware missing-data behavior, availability-reliability gap,
explainable priority ranking, isolated what-if scenarios, provenance-aware village profiles, GIS map with
table fallback, methodology page and CSV export. Keep scoring formulas in backend domain modules, not UI
components or route handlers. Add tests for all acceptance criteria. Use environment variables for secrets.
Do not add optional AI features until the full MVP passes tests. Work in small milestones, run tests, and
report files changed, tests executed, failures and next task. If a data source or requirement is ambiguous,
document the assumption and preserve data provenance rather than fabricating facts.
```

12. Release gate

• All P0 tasks completed.

• Unit, API and end-to-end critical path tests pass.

• Deployed application works in a clean browser session.

• All sample records are labelled as demo/simulated.

• Source register and limitations are visible.

• Team can perform the demo without manually editing the database.

• Known limitations are documented; no unsupported “AI prediction” claim is used.

13. Research links

Gram Manchitra | Mission Antyodaya | BharatNet | TRAI MySpeed | World Bank Rural Access Index

These sources inform the existing-system landscape; availability of an API, downloadable dataset or permission to reuse data must be verified separately before integration.